

LA DECISIÓN EN LA ESTRUCTURA

K. V Ramirez Ambrocio

7690-22-2239 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

kramireza10@miumg.edu.gt

Resumen

El objetivo es analizar los criterios que debe considerar un desarrollador al momento de elegir la arquitectura de software más adecuada según el tipo de aplicación que se va a construir. Se hizo a partir de la revisión de sitios y artículos sobre atributos de calidad en arquitectura de software, así como en la comparación de los tres estilos arquitectónicos más utilizados actualmente, como son monolítico, microservicios y serverless. Como resultado, se demostró que la elección correcta no depende de seguir la tendencia más popular del momento, sino de evaluar puntos concretos, como el presupuesto disponible, la carga esperada y las necesidades del sistema. Llegando a la conclusión de que no existe una arquitectura mejor, sino una arquitectura que mejor se adecúa para cada proyecto y decidir a partir de las necesidades, en lugar de modas del mercado así reducimos el riesgo de cometer un mal uso de los recursos en un proyecto pequeño o de quedarse corto frente a un proyecto demasiado grande por sus requerimientos o por personal.

Palabras clave: arquitectura de software, monolito, microservicios, serverless, atributos de calidad.

Desarrollo del tema

Elegir la arquitectura de una aplicación es una de las decisiones más importantes dentro de un proyecto de software, porque a diferencia de otras decisiones, esta es difícil de revertir una vez que el proyecto ya está empezado. La arquitectura de un sistema no se trata solo de cómo se organiza el código, sino de cómo esa organización hace que el sistema cumpla con sus atributos de calidad, como el rendimiento, la seguridad, la disponibilidad y la facilidad de mantenimiento. Esto significa que antes de elegir un estilo arquitectónico, hay que tener claro qué es lo que realmente le importa al proyecto, preguntarse como ¿necesita responder rapidísimo? ¿necesita estar disponible todo el tiempo? ¿necesita ser fácil de modificar por un equipo grande? Cada arquitectura resuelve mejor unos atributos que otros, y ninguna los resuelve todos al mismo tiempo.

El primer estilo, y el más tradicional, es la arquitectura monolítica. Aquí toda la aplicación vive en una sola base de código y se despliega como una sola unidad. La principal ventaja es la simplicidad es decir, más fácil de desarrollar, probar y desplegar en las primeras etapas de un proyecto, sobre todo si se trata de un producto pequeño o de una aplicación pequeña con una carga de trabajo predecible. El problema aparece cuando la aplicación crece, si una sola función del sistema necesita más recursos, no se puede escalar solo esa parte, hay que escalar toda la aplicación completa, lo cual termina siendo caro e ineficiente.

El segundo estilo son los microservicios, donde la aplicación se divide en varios servicios pequeños e independientes, cada uno responsable de una parte especializado en hacer alguna funcionalidad. Esta arquitectura resuelve justamente el problema de escalabilidad del monolito, porque cada servicio se puede escalar por separado según su propia necesidad. También permite que equipos

distintos trabajen al mismo tiempo sin pisarse el trabajo entre sí, lo cual es una ventaja enorme cuando el equipo de desarrollo es grande. Sin embargo, esta arquitectura no es gratis en términos de complejidad es decir, ahora hay que coordinar la comunicación entre servicios, manejar fallas de red, y mantener consistencia de datos entre partes que antes vivían juntas en una sola base de datos. Por eso se recomienda para proyectos grandes y complejos, no para aplicaciones pequeñas que apenas están arrancando.

El tercer estilo es serverless, donde en lugar de mantener servidores corriendo todo el tiempo, el código se ejecuta en funciones que el proveedor de la nube activa solo cuando ocurre un evento específico. La ventaja principal es el costo solo se paga por el tiempo de cómputo que realmente se usa y no hay que preocuparse por administrar servidores. Esto lo hace mejor para tareas que ocurren muy pocas veces, como procesar una notificación o para aplicaciones con picos de tráfico muy impredecibles. La desventaja más conocida es lo tardado de iniciar todo la primera vez, que ocurre cuando una función no se ha usado en un tiempo y tarda un poco más en responder la primera vez que se llama.

Con estos tres estilos claros, la pregunta real no es cuál arquitectura es la mejor, sino cuál le conviene más al tipo de aplicación que se está construyendo. Para un producto pequeño funcional o una aplicación pequeña con pocos usuarios, lo más recomendable suele ser un monolito, porque permite avanzar rápido sin cargar con los problemas de coordinar servicios que todavía no se necesitan. Para una aplicación grande, con mucho tráfico constante y con un equipo de desarrollo dividido en varios grupos, los microservicios tienen más sentido, porque permiten escalar cada parte del sistema según su propia necesidad. Y para funcionalidades puntuales, con carga de trabajo dividido o poco frecuente, como el envío de correos, el procesamiento de imágenes o las notificaciones, serverless suele ser la opción más eficiente en costo.

Es importante mencionar que estas tres arquitecturas pueden trabajar juntas. En la práctica, muchos sistemas actuales combinan un monolito para la lógica central del negocio con funciones serverless para tareas específicas orientadas a eventos, como recordatorios o procesamiento en segundo plano. Esto demuestra que la decisión arquitectónica no siempre es elegir una sola opción para todo el sistema, sino identificar qué parte de la aplicación necesita qué tipo de solución.

También vale la pena aclarar la diferencia entre información sólida y contenido de marketing en este tema. Mucho de lo que circula en blogs técnicos presenta a los microservicios como la opción "moderna" al monolito como algo obsoleto, pero esa narrativa no está respaldada por evidencia técnica, insiste en que la elección debe basarse en los requerimientos y en la calidad que el proyecto necesita, no en la popularidad de un estilo arquitectónico en un momento dado.

Observaciones y comentarios

Algo que llama la atención al investigar este tema es que muchos equipos de desarrollo adoptan microservicios desde el inicio de un proyecto, sin tener todavía la carga de trabajo ni el tamaño de equipo que justificaría esa complejidad. Esto termina generando lo que se conoce como un "monolito distribuido".^{es} decir un sistema que tiene toda la complejidad de los microservicios, pero ninguno de sus beneficios reales, porque los servicios siguen dependiendo unos de otros de forma muy acoplada.

Otra observación es que la decisión de arquitectura casi nunca se explica como por ejemplo, cuánta

disponibilidad necesita el sistema o qué tan rápido debe responder, sino en términos de moda tecnológica. Esto es preocupante porque una decisión tan costosa de revertir debería basarse en necesidades reales del proyecto y no en lo que otras empresas están usando.

Conclusiones

1. La arquitectura de software debe elegirse en función de lo que el proyecto necesita y no según la tendencia tecnológica más popular del momento.
2. La arquitectura monolítica sigue siendo la opción más adecuada para aplicaciones pequeñas con carga predecible, por su simplicidad de desarrollo y despliegue.
3. Los microservicios resuelven el problema de escalabilidad del monolito, pero introducen una complejidad de coordinación que solo se justifica en proyectos grandes con equipos de desarrollo igualmente grandes.
4. La arquitectura serverless resulta más eficiente en costo para tareas puntuales, casi nunca van a pasar o con carga de trabajo muy variable, aunque presenta limitaciones como la latencia de arranque en frío.
5. Ninguna arquitectura es universalmente superior y muchos sistemas actuales combinan varios estilos arquitectónicos dentro de una misma aplicación, según lo que cada parte del sistema realmente necesita.

Referencias

- Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.). Addison-Wesley.
- Calmops. (2026, 18 de marzo). *Monolithic vs microservices vs serverless: A practical guide for 2026*. <https://calmops.com/architecture/monolithic-vs-microservices-vs-serverless-2026-practical-guide/>
- Kalra, S. (2025, 21 de septiembre). *Understanding key software architecture patterns: Monolith, microservices, monorepo, and more*. DEV Community. <https://dev.to/sahilkalra/understanding-key-software-architecture-patterns-monolith-microservices-monorepo-and-more-350f>
- Kruscheva Company. (2024, 30 de septiembre). *Microservices vs monolith: Best architectural strategy*. <https://kruscheva.com/microservices-vs-monolith-best-architectural-strategy/>
- Repositorio Git: <https://github.com/Valkill/seminario-encuesta.git>